Edge Intelligence LAB

Pradyumna Paithankar

25MML0035

Frame: It is a 3D matrix with pixel values.

1) Canny math behind edge:

1. Gaussian Smoothing (Noise Reduction)

Before detecting edges, noise must be removed because gradients amplify noise.

The image $I(x, y)$is convolved with a Gaussian kernel.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Smoothed image: $$I_s(x, y) = I(x, y) * G(x, y)$$

Where, σ= standard deviation controlling blur

2. Gradient Computation

Edges correspond to large intensity changes.

OpenCV typically uses Sobel operators to approximate derivatives.

Gradient in x-direction: $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

Gradient in y-direction: $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

Compute derivatives: $I_x = I_s * G_x$

$I_y = I_s * G_y$

Gradient Magnitude:

Edge strength: $$M(x, y) = \sqrt{I_x^2 + I_y^2}$$

Often approximated for speed: $M(x,y) \approx |I_x| + |I_y|$

Gradient Direction: $\theta(x,y) = \tan^{-1}\left(\frac{I_y}{I_x}\right)$

Direction tells which way the edge runs.

Angles are usually quantized into 4 directions:

- 0°
- 45°
- 90°
- 135°

3. Non-Maximum Suppression

This step thins edges to 1-pixel width.

For each pixel:

1. Look along gradient direction
2. Compare magnitude with neighbouring pixels
3. Keep pixel only if it is a local maximum

Mathematically: $M(x,y) = \begin{cases} M(x,y) & \text{if } M(x,y) > M_1 \text{ and } M(x,y) > M_2 \\ 0 & \text{otherwise} \end{cases}$

Where M_1, M_2 are magnitudes of neighboring pixels along gradient direction.

4. Double Thresholding

Pixels are classified using two thresholds:

- T_H = High threshold
- T_L = Low threshold

Strong edge if $M(x,y) \geq T_H$
Weak edge if $T_L \leq M(x,y) < T_H$
Non-edge if $M(x,y) < T_L$

5. Edge Tracking by Hysteresis

Weak edges are kept only if connected to strong edges.

If a weak edge pixel p is 8-connected to a strong edge pixel, then $p \rightarrow$ edge
Otherwise, $p \rightarrow 0$
This prevents broken edges.

2) Math behind negative/inverted image:

Let:

- r= original pixel intensity
- s= transformed (output) pixel intensity
- L= total number of possible intensity levels

The transformation is: $s = (L - 1) - r$

For an 8-bit Image

Most grayscale images are 8-bit, meaning: $L = 256$

So, the formula becomes: $s = 255 - r$

3) Math behind motion detection:

1. Representing Video Frames Mathematically

A video is a sequence of images over time. $F(x, y, t)$
Where:

- x, y→ pixel location
- t→ time (frame number)
- $F(x, y, t)$→ intensity of pixel at that time

Frame 1 → $F(x, y, t)$
Frame 2 → $F(x, y, t + 1)$

2. Frame Differencing (Core Motion Detection Math)

Motion is detected by subtracting two frames.

$$D(x, y) = | F(x, y, t + 1) - F(x, y, t) |$$

Where:

- $D(x, y)$= difference image
- absolute value ensures positive change

If the pixel did not move, the value will be close to 0. If the pixel changed due to motion, the value will be large.

Example:

Pixel	**Frame t**	**Frame t+1**	**Difference**
A	120	121	1
B	50	200	150

- A → no motion
- B → motion detected

3. Thresholding

Not all differences indicate motion (camera noise, lighting). So we apply a threshold T.

$$M(x, y) = \begin{cases} 1 & \text{if } D(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

Where:

- $M(x, y)$= motion mask
- 1 = motion
- 0 = background

4. Motion Region Detection

After thresholding, we get a binary image: $M(x, y)$

Connected pixels are grouped into moving objects using connected components.

$$A = \sum_{x,y} M(x, y)$$

If area A is above some minimum value → object detected.

➔ Motion detection relies on the temporal derivative of image intensity.
If this derivative is non-zero, motion exists.